Reflected XSS into HTML Context with Most Tags and Attributes Blocked

1. Overview

- **Vulnerability:** Reflected Cross-Site Scripting (XSS)
- **Category:** Web Application Security
- **Platform:** PortSwigger Web Security Academy
- **Severity:** Context-dependent
- **OWASP Category:** A03:2025 – Injection

2. Description

Reflected Cross-Site Scripting (XSS) occurs when an application includes user-controlled input in an HTTP response without applying appropriate output encoding or sanitization.

In this lab, most HTML tags and attributes are blocked by input filters. However, an alternative HTML element and event handler may bypass the filtering restrictions, allowing JavaScript execution in the victim's browser.

3. Vulnerability Details

The application reflects user-supplied input into an HTML context. Although a filter attempts to block common tags and attributes, incomplete filtering can allow an attacker to introduce executable markup.

Affected component: The input parameter used by the lab's search functionality.

Root cause: Reliance on incomplete HTML tag and attribute filtering rather than context-aware output encoding.

4. Proof of Concept (PoC)

1. Open the assigned PortSwigger lab.
2. Identify the input field that reflects user-controlled content.
3. Inspect the response to determine how the input is inserted into the HTML document.
4. Use the lab's built-in exploit guidance to identify an allowed HTML element and event handler.

5. Verify whether the resulting payload executes JavaScript in the lab environment.

Example harmless test payload, if the relevant tag and event handler are permitted:

<iframe src="javascript:alert(1)">

The example is illustrative; the lab's filtering rules may block it. Use the actual successful payload observed during your authorized lab testing as the final PoC.

5. Impact

If exploited against a real application, reflected XSS may allow an attacker to:

- Execute JavaScript in a victim's browser under the vulnerable site's origin.
- Read or modify accessible page content.
- Perform actions as the victim where application protections permit.
- Present convincing phishing content within the trusted website.

The actual impact depends on the application's authentication, browser protections, and accessible data.

6. Remediation

- Apply context-aware output encoding before inserting untrusted data into HTML.
- Avoid relying on blocklists of HTML tags and attributes.
- Use a well-maintained HTML sanitization library when user-supplied HTML must be supported.
- Prefer safe DOM APIs such as `textContent` instead of unsafe HTML insertion.
- Deploy Content Security Policy (CSP) as an additional defense-in-depth measure.
- Add regression tests for alternative tags, event handlers, and filter bypasses.

7. References

- PortSwigger Web Security Academy: https://portswigger.net/web-security/cross-site-scripting
- OWASP Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

8. Disclaimer

This report documents an authorized educational lab scenario. Testing should only be performed on systems for which explicit permission has been granted.